

Homework 4

Daniel Nkunga

1. Describe the purpose of the homework and what you did / what you implemented.

This assignment covered a variety of topics mostly regarding sorting algorithms of various kinds. The typical sorting algorithms were quick and merge sorts, two of the most popular sorting algorithms for their speed and efficiency. A red-black tree was also implemented to finish the discussion of trees as it mainly serves as an additional layer of rules for a greater tree class. The final set of programs was related to graphs. Prim's, Kruskal's, and Dijkstra's algorithms are all used to produce a minimum-spanning tree from a graph, the most efficient version of said graph. Topological sort is a method of displaying a directed graph where nodes will not appear before their future neighbor.

The assignment was larger than previous ones and very scattered, but also very interesting to implement.

2. Describe your approach and explain how you solved the problem (the key ideas for implementing each required function).

Merge sort is a sorting algorithm with a time complexity of $O(n \log n)$. It is based on the idea that it is easier to sort two sorted list than it is to an entirely unsorted list. It does this by recursively breaking the list in half and checking to see if that list is sorted. If a list only consists of one element, then it is considered sorted. Once it has two sorted list, it can quickly merge the list together by comparing the top element and adding them to a new list.

The function itself consist of two functions, merge and merge sort. The merge sort function recursively calls the merge algorithm on itself, breaking each list in half until it has a sorted list. The merge algorithms perform the logic of merging two sorted lists together. The only optimization in the merge algorithm is that it will stop comparing if one of the list runs out of elements.

Heap sort is a sorting algorithm with a time complexity of $O(n \log n)$. It is based on the idea of a heap, a structure where each node is bigger than its children. It does this by recursively calling a "heapify" algorithm on smaller portions of the list until the entire thing is sorted.

Heap sort consist of two functions, heapify and heap sort. The heap sort function recursively calls the heapify function on half of the list and then the entire list to properly ensure the max element will end up at the end of the final array. The heapify function establishes the largest element in the heap and swaps it with and larger children to the left of it.

Topological sort is a sorting algorithm for directed graphs that produces an ordering of nodes such that for every directed edge from node A to node B, node A appears before node B in the result. It is based on the idea of in-degrees, which is the number of edges pointing into a node. A node with an in-degree of zero has no

prerequisites and is safe to place next in the order. The algorithm begins by calculating the in-degree of every node in the graph. Any node with an in-degree of zero is added to a queue. It then processes each node in the queue, adding it to the result and reducing the in-degree of its neighbors. If any neighbor's in-degree drops to zero, it is added to the queue. This continues until the queue is exhausted. If any nodes were never processed, it means a cycle exists in the graph and no valid topological ordering is possible.

Prim's algorithm is a method for finding a minimum spanning tree in a graph. It does this by starting at an arbitrary location on the graph and choosing to explore to its neighbors least cost. It will then explore from both the initial node and choose the cheapest option again. This process will continue until all nodes are explored and added to this new tree. The algorithm first finds all neighbors that can be visited and adds them to a heap of places that can be explored. It compares this heap to find the cheapest cost and adds that neighbor into the minimum spanning tree. The new neighbor then has its neighbors added to the heap of places that can be explored and is removed from the heap itself. This process continues until the heap exhaust itself.

Kirskal's algorithm is a method for finding a minimum spanning tree by adding the lowest cost connections to the minimum spanning tree so long as they do not form a cycle. It will look at every possible connection, choose the cheapest, check for a cycle (and remove it if it does form a cycle), then add it to the minimum spanning tree. The function first gets then sorts all of the edges. It will then try and add an edge to the tree so long as the tree has not reached every node. If a cycle is not found, that edge is added to the minimum spanning tree and the edge is tracked to make sure it will not be part of a future cycle.

Dijkstra's algorithm is a method for finding a minimum spanning tree from one given starting location. It does this by getting the shortest path from the starting node to its neighbors and choosing the cheapest option. It will then explore that neighbor's neighbors by adding the cost to get there through the neighbor. This iterates from node to node until the entire tree is explored. If at any point there exist a cheaper path from the start to a node, that path is kept instead of the previous path. The algorithm first establishes the cost from the starting node to every neighbor as the maximum possible value. If the neighbors visited, its cost is modified and its trace back to the start is memorized. This continues until every node is discovered. Afterwards, minimum spanning tree is constructed by stringing together the path from the start to each node by following the trace set by each node.

Red-black trees are a variation of binary search trees that keep themselves balanced by assigning each node a color, either red or black, and enforcing a set of rules about how those colors can be arranged. This prevents the tree from becoming lopsided, which guarantees a time complexity of $O(\log n)$ for insertions and lookups. Every new node is inserted as red. If that causes a violation, the tree fixes itself through a combination of recoloring and rotations. A rotation is an operation that

restructures a portion of the tree by pivoting around a node, either to the left or to the right, without breaking the ordering of the tree. The rules the tree enforces are that the root must always be black, no two red nodes can appear in a row on any path, and every path from the root to a leaf must pass through the same number of black nodes. After every insertion, the fix violation function walks back up the tree and resolves any broken rules. If a node's parent and uncle are both red, the fix is simply to recolor them black and the grandparent red, then continue checking upward. If the uncle is black, a rotation is needed to restructure that portion of the tree before recoloring can restore balance.

3. Discuss any bugs or problems you encountered and how you fixed them.

The assignment went pretty smoothly! The few major issues I encountered came with merge sort. I initially didn't do proper edge handling was initially running over the end of the array when trying to sort the list in main.

4. Include screenshots or images of your results.

I reset my laptop over break and wasn't able to get OpenCV running again. Here is a copy of the terminal.

```
[daniel_n@akihayakawa C++HW4_Sourcecode]$ make

Perform unit test of the Dijkstra algorithm
Path from 0 to 5: 0 -> 1 -> 2 -> 4 -> 5

Path from SAEF to GRAD: SAEF -> BUCH -> LCAF -> GLAD2 -> PDTF -> SINIF -> LMK111 -> WAHR -> SDPG2 -> ARKU2 -> POST -> FPAC -> WATR -> ARSAGAS -> CORD1 -> PTSC -> MEMH2 -> ARMY -> GRAD
You have to use OpenCV to visualize your map road

Perform unit test of the Red-Black Tree

=====
--- STARTING INTEGRATED ALGORITHM STACK DEMO ---
=====
[Step 1] RB-Tree: Coordinates stored for Buildings 0, 1, and 2.
-> Found Building 0 at (100, 100)
-> Found Building 1 at (200, 300)
-> Found Building 2 at (400, 150)

[Step 2] Graph: Edges created with Euclidean weights:
-> Edge (0,1) weight: 223
-> Edge (1,2) weight: 250
-> Edge (0,2) weight: 304

[Step 3] Kruskal: Constructing Minimum Spanning Tree...
-> Added Edge (0,1) w:223
-> Added Edge (1,2) w:250
Total MST Cost: 473

[Step 4] Dijkstra: Calculating shortest path from 0 to 2...
-> Shortest Path: 0 -> 2

[Step 5] HeapSort: Sorting traffic congestion scores...
-> Original: 85 12 44 98 30
-> Sorted: 12 30 44 85 98

=====
--- INTEGRATED DEMO COMPLETE ---
=====
[daniel_n@akihayakawa C++HW4_Sourcecode]$
```

Perform unit test of the Dijkstra algorithm

Path from 0 to 5: 0 -> 1 -> 2 -> 4 -> 5

Path from SAEF to GRAD: SAEF -> BUCH -> LCAF -> GLAD2 -> PDTF -> SINIF -> LMK111 -> WAHR -> SDPG2 -> ARKU2 -> POST -> FPAC -> WATR -> ARSAGAS -> CORD1 -> PTSC -> MEMH2 -> ARMY -> GRAD

You have to use OpenCV to visualize your map road

Perform unit test of the Red-Black Tree

=====

--- STARTING INTEGRATED ALGORITHM STACK DEMO ---

=====

[Step 1] RB-Tree: Coordinates stored for Buildings 0, 1, and 2.

-> Found Building 0 at (100, 100)

-> Found Building 1 at (200, 300)

-> Found Building 2 at (400, 150)

[Step 2] Graph: Edges created with Euclidean weights:

-> Edge (0,1) weight: 223

-> Edge (1,2) weight: 250

-> Edge (0,2) weight: 304

[Step 3] Kruskal: Constructing Minimum Spanning Tree...

-> Added Edge (0,1) w:223

-> Added Edge (1,2) w:250

Total MST Cost: 473

[Step 4] Dijkstra: Calculating shortest path from 0 to 2...

-> Shortest Path: 0 -> 2

[Step 5] HeapSort: Sorting traffic congestion scores...

-> Original: 85 12 44 98 30

-> Sorted: 12 30 44 85 98

=====

--- INTEGRATED DEMO COMPLETE ---

=====